DBMS TASK 9

Multi-Table Reports with Joins and Subqueries

Purpose

Build reports that combine related records across OnlineShoppingDB and use subqueries to compare individual values with calculated summaries.

A. Detailed order lines

```
SELECT o.OrderID, c.CustomerName, p.ProductName,
       d.Quantity, d.UnitPrice,
       d.Quantity * d.UnitPrice AS LineTotal
FROM Orders AS o
JOIN Customers AS c ON c.CustomerID = o.CustomerID
JOIN OrderDetails AS d ON d.OrderID = o.OrderID
JOIN Products AS p ON p.ProductID = d.ProductID
ORDER BY o.OrderID, d.OrderDetailID;
```

Report use: Combines four tables to show who ordered each item and the calculated line amount.

B. Order status with payment details

```
SELECT o.OrderID, c.CustomerName, o.OrderDate, o.Status,
       p.PaymentMethod, p.Amount, p.PaymentStatus
FROM Orders AS o
JOIN Customers AS c ON c.CustomerID = o.CustomerID
LEFT JOIN Payments AS p ON p.OrderID = o.OrderID
ORDER BY o.OrderID;
```

Report use: Shows customer, order, and payment information together. LEFT JOIN keeps orders even when no payment row exists.

C. Order value by customer

```
SELECT c.CustomerID, c.CustomerName,
       COALESCE(SUM(d.Quantity * d.UnitPrice), 0) AS TotalOrderValue
FROM Customers AS c
LEFT JOIN Orders AS o ON o.CustomerID = c.CustomerID
LEFT JOIN OrderDetails AS d ON d.OrderID = o.OrderID
GROUP BY c.CustomerID, c.CustomerName
ORDER BY TotalOrderValue DESC;
```

Report use: Calculates the value of order lines for each customer, including customers without orders.

D. Find products without sales

```
SELECT p.ProductID, p.ProductName
FROM Products AS p
LEFT JOIN OrderDetails AS d ON d.ProductID = p.ProductID
WHERE d.OrderDetailID IS NULL;
```

Report use: Finds catalogue items that have never appeared on an order line. The provided sample should return no rows.

E. Customers with repeat orders

```
SELECT c.CustomerID, c.CustomerName,
       COUNT(o.OrderID) AS OrderCount
FROM Customers AS c
JOIN Orders AS o ON o.CustomerID = c.CustomerID
GROUP BY c.CustomerID, c.CustomerName
HAVING COUNT(o.OrderID) > 1;
```

Report use: Lists customers who have more than one order; Arun Prabu has two in the sample.

F. Products above the average price

```
SELECT ProductName, Price
FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products)
ORDER BY Price DESC;
```

Report use: The inner query calculates the average product price; the outer query lists products above it.

G. Payments above the mean amount

```
SELECT PaymentID, OrderID, PaymentMethod, Amount
FROM Payments
WHERE Amount > (SELECT AVG(Amount) FROM Payments)
ORDER BY Amount DESC;
```

Report use: Compares each payment with the average payment amount.

H. Sales value by category

```
SELECT p.Category,
       SUM(d.Quantity * d.UnitPrice) AS CategorySales
FROM Products AS p
JOIN OrderDetails AS d ON d.ProductID = p.ProductID
GROUP BY p.Category
ORDER BY CategorySales DESC;
```

Report use: Summarizes order-line value for each product category.

Evidence to Include

For each report, run the SQL in MySQL Workbench and add a screenshot that shows the query and the corresponding Result Grid. Keep the column labels visible; if the output is long, capture the most relevant rows.

Closing Remarks

Joins bring related information into one report, while subqueries provide comparison values such as averages. Together, these techniques support analysis of customer activity, product sales, orders, and payment status.

SQL CODE — MYSQL WORKBENCH

Run each report separately and add screenshots of the result grids.

```
USE OnlineShoppingDB;

-- 1. Order lines with customer and product names
SELECT o.OrderID, c.CustomerName, p.ProductName,
       d.Quantity, d.UnitPrice,
       d.Quantity * d.UnitPrice AS LineTotal
FROM Orders o
JOIN Customers c ON c.CustomerID=o.CustomerID
JOIN OrderDetails d ON d.OrderID=o.OrderID
JOIN Products p ON p.ProductID=d.ProductID
ORDER BY o.OrderID, d.OrderDetailID;

-- 2. Orders and payments
SELECT o.OrderID, c.CustomerName, o.OrderDate, o.Status,
       p.PaymentMethod, p.Amount, p.PaymentStatus
FROM Orders o
JOIN Customers c ON c.CustomerID=o.CustomerID
LEFT JOIN Payments p ON p.OrderID=o.OrderID
ORDER BY o.OrderID;

-- 3. Total order-line value by customer
SELECT c.CustomerID, c.CustomerName,
       COALESCE(SUM(d.Quantity*d.UnitPrice),0) AS TotalOrderValue
FROM Customers c
LEFT JOIN Orders o ON o.CustomerID=c.CustomerID
LEFT JOIN OrderDetails d ON d.OrderID=o.OrderID
GROUP BY c.CustomerID, c.CustomerName
ORDER BY TotalOrderValue DESC;

-- 4. Products never ordered
SELECT p.ProductID, p.ProductName
FROM Products p LEFT JOIN OrderDetails d ON d.ProductID=p.ProductID
WHERE d.OrderDetailID IS NULL;

-- 5. Customers with multiple orders
SELECT c.CustomerID, c.CustomerName, COUNT(o.OrderID) AS OrderCount
FROM Customers c JOIN Orders o ON o.CustomerID=c.CustomerID
GROUP BY c.CustomerID, c.CustomerName
HAVING COUNT(o.OrderID)>1;

-- 6. Products above average price
SELECT ProductName, Price FROM Products
WHERE Price > (SELECT AVG(Price) FROM Products)
ORDER BY Price DESC;

-- 7. Payments above average amount
SELECT PaymentID, OrderID, PaymentMethod, Amount FROM Payments
WHERE Amount > (SELECT AVG(Amount) FROM Payments)
ORDER BY Amount DESC;

-- 8. Sales value by category
SELECT p.Category, SUM(d.Quantity*d.UnitPrice) AS CategorySales
FROM Products p JOIN OrderDetails d ON d.ProductID=p.ProductID
GROUP BY p.Category ORDER BY CategorySales DESC;
```